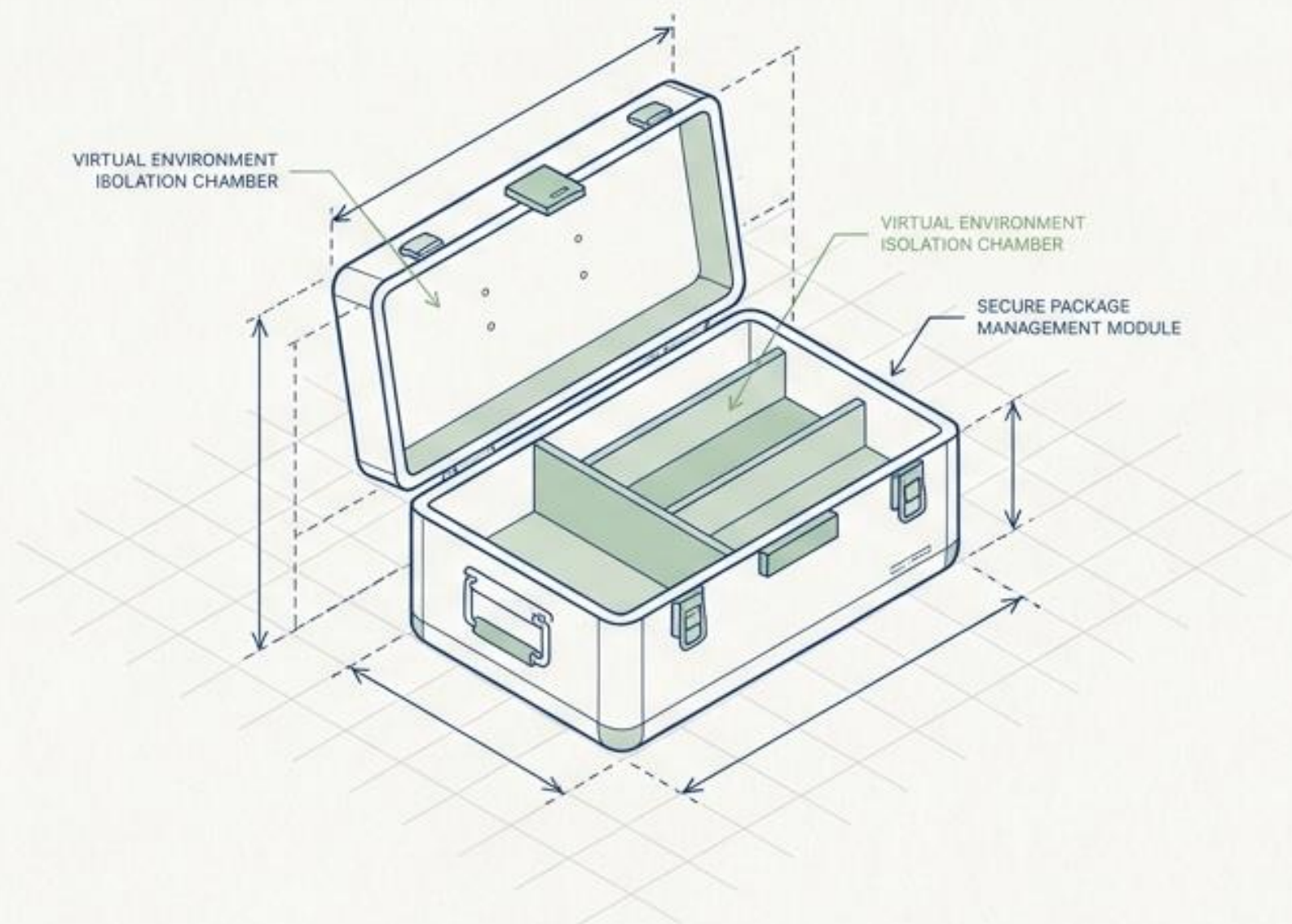


안전한 코딩을 위한 첫걸음

미니콘다(Miniconda)와 가상환경 완벽 이해



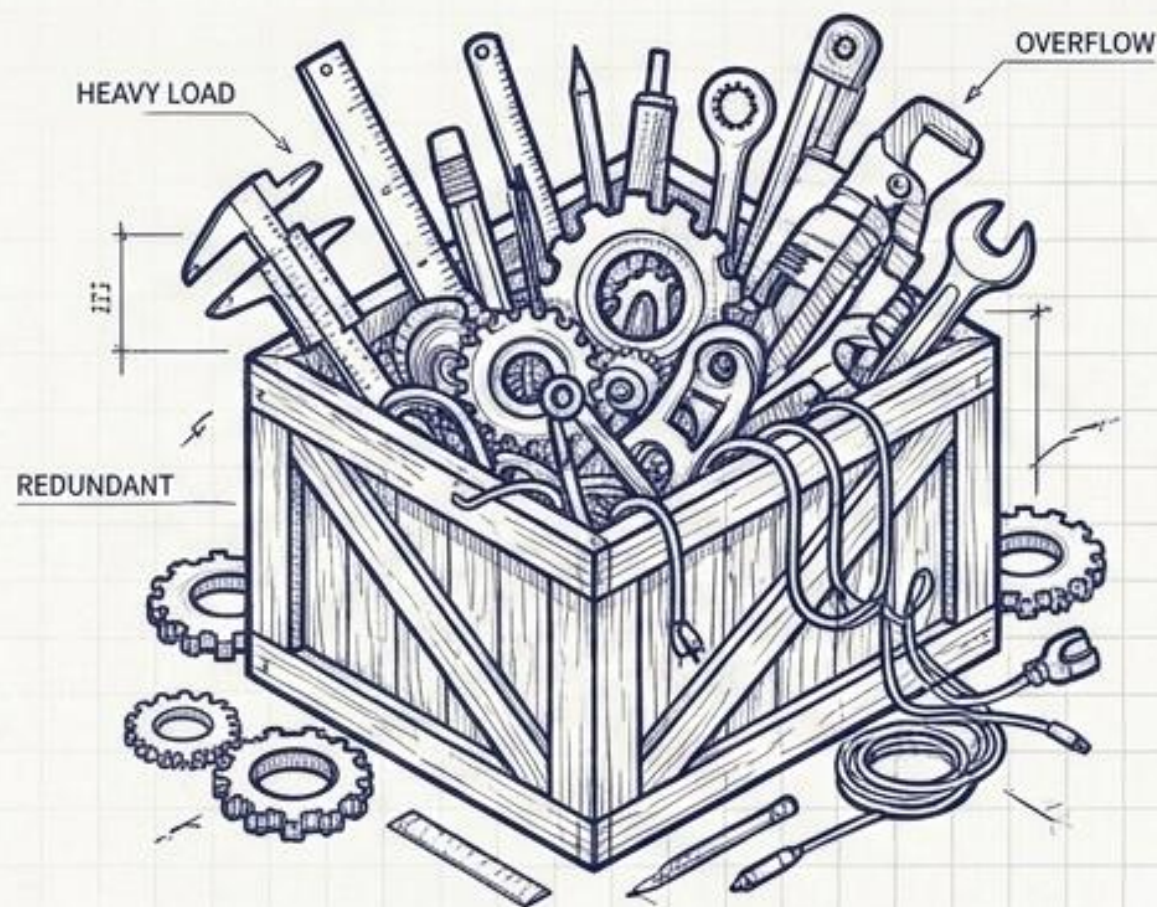
패키지 하나 잘못 깔았다가 내 컴퓨터 시스템이 꼬여버리면 어떡하지?

파이썬 개발 환경을 처음 세팅할 때 누구나 겪는 근본적인 두려움입니다.
기존 시스템의 도구들과 새로 설치하는 패키지들이 충돌하면 OS 전체가 망가질 수 있습니다.



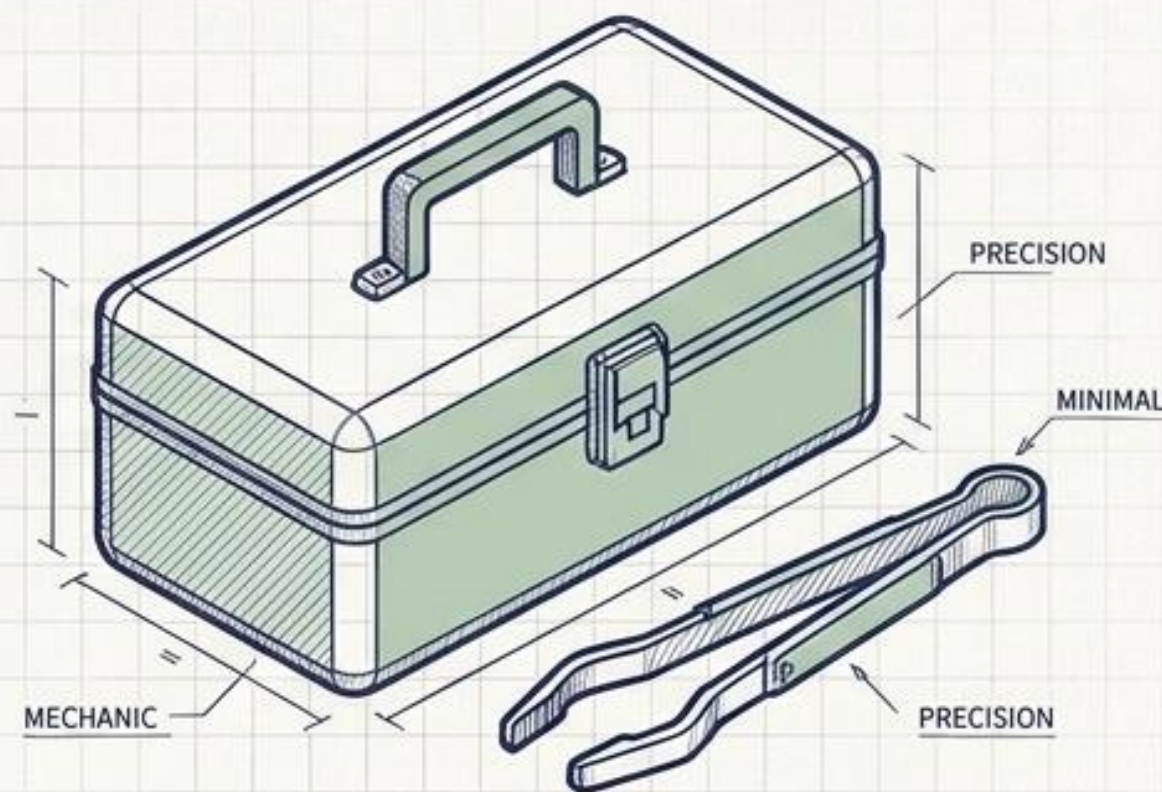
도구의 선택: 왜 아나콘다가 아닌 미니콘다인가?

아나콘다 (Anaconda)



개념: 온갖 연구와 프로그램이 다 들어있는 종합 선물 세트
용량: 수 GB (매우 무거움)
특징: 안 쓰는 도구까지 모두 설치됨

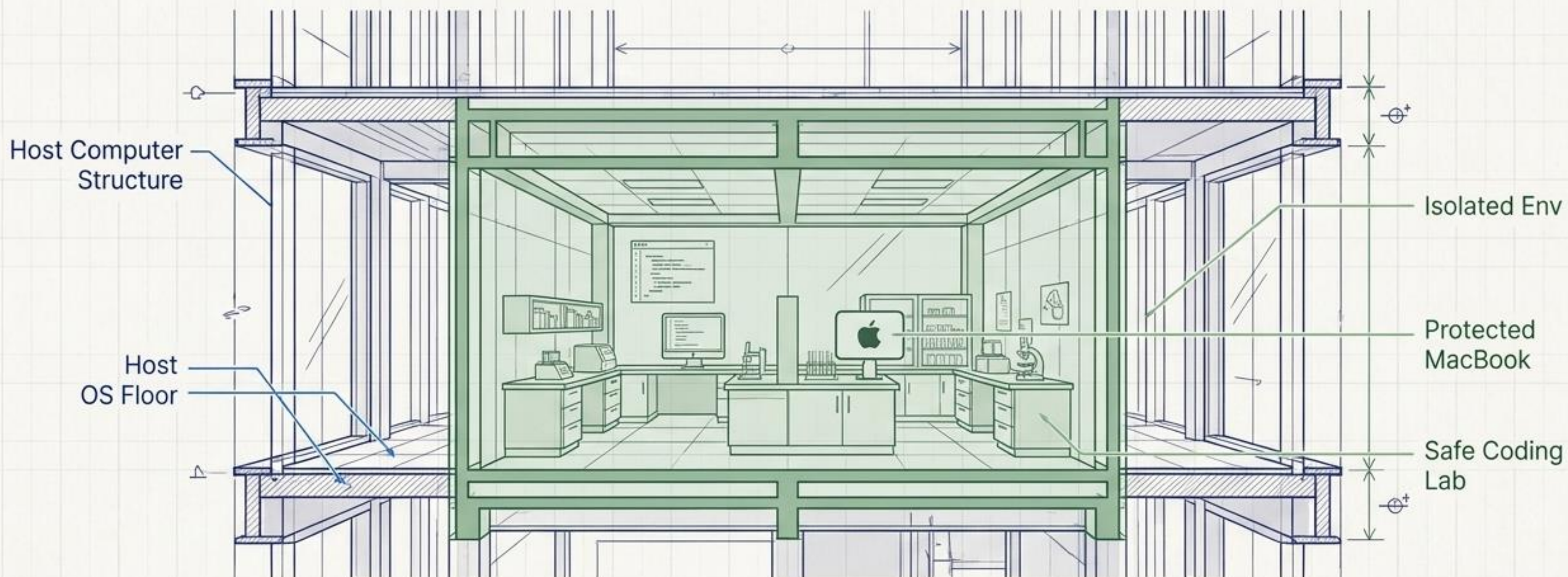
미니콘다 (Miniconda)



개념: 빈 공구함 + 핵심 집게
용량: 최소 용량 (가볍고 빠름)
특징: 필요한 라이브러리만 깔끔하게 설치 및 완벽한 통제 가능

가상환경: 내 컴퓨터 안의 '독립된 방(실습실)'

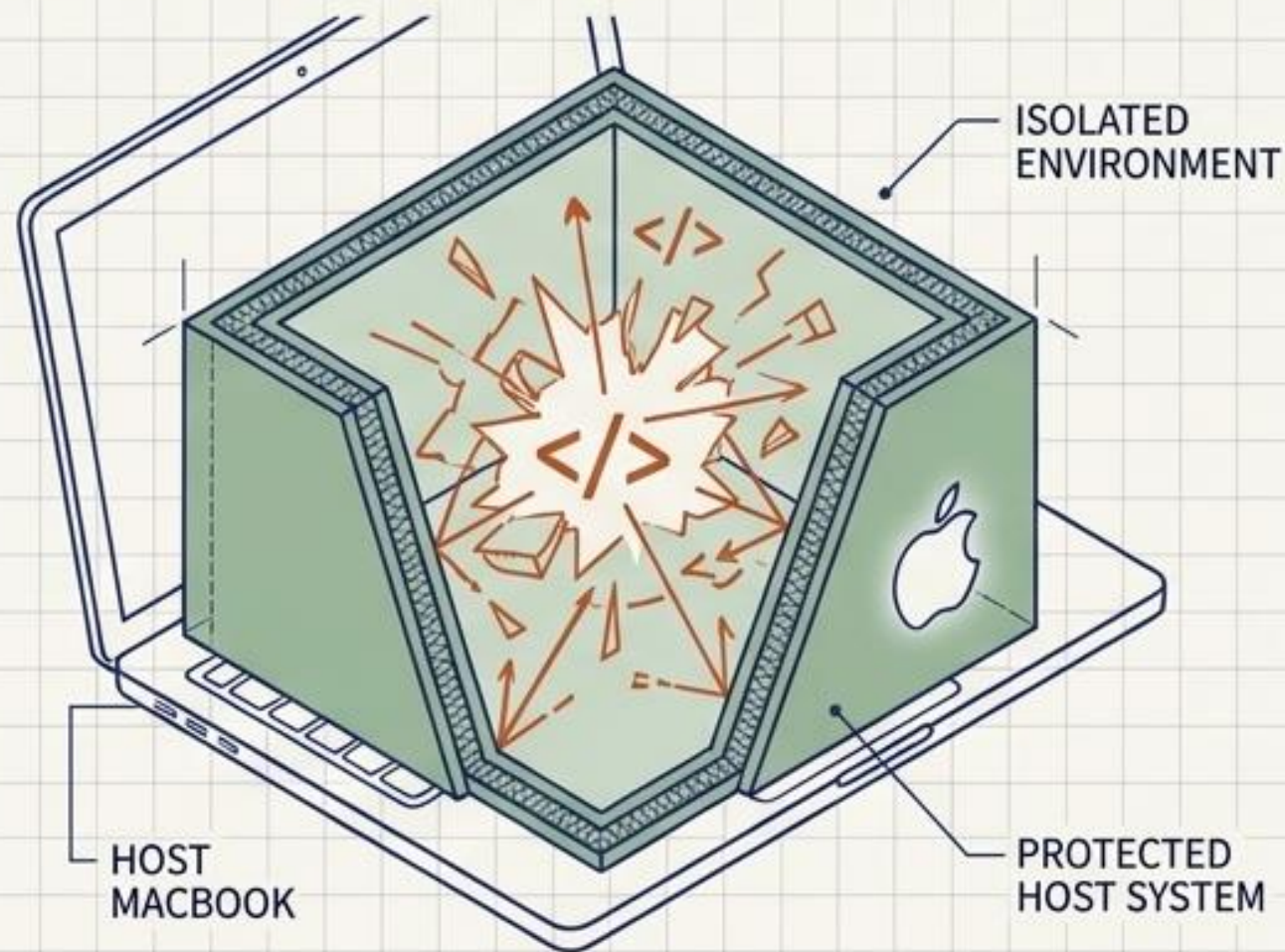
'Conda 가상환경'을 만든다는 것은 컴퓨터 안에 기존 시스템과 완전히 분리된 독립적인 방을 하나 짓는 개념입니다. 맥북을 안전하게 보호하면서 마음 놓고 코딩할 수 있는 나만의 전용 실습실을 준비하는 과정입니다.



가상환경이 제공하는 2가지 절대적 이점

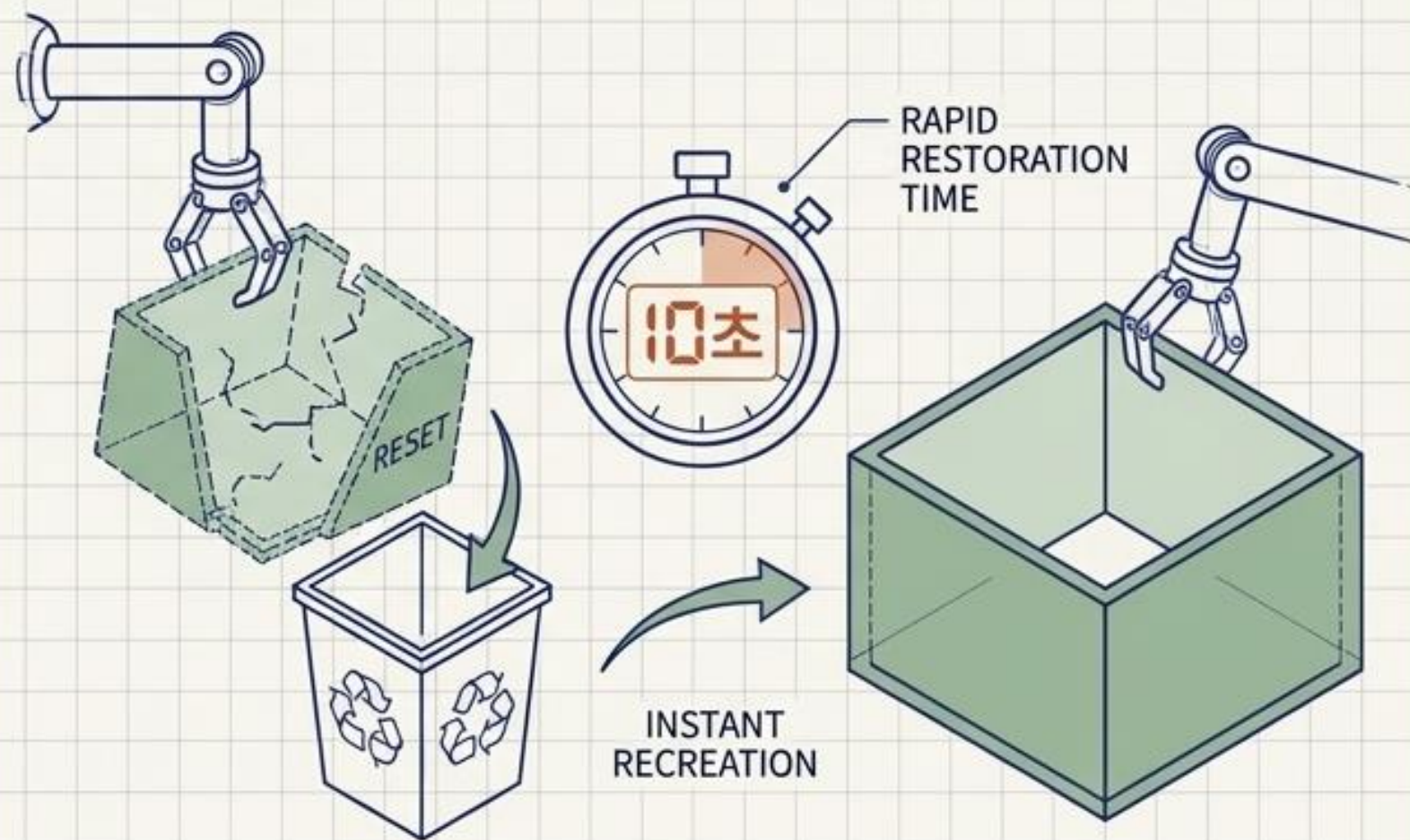
완벽한 시스템 보호

이 방 안에 무엇을 설치하고 실험해도,
방 바깥의 맥북 기본 시스템에는 전혀 영향을 주지 않습니다.



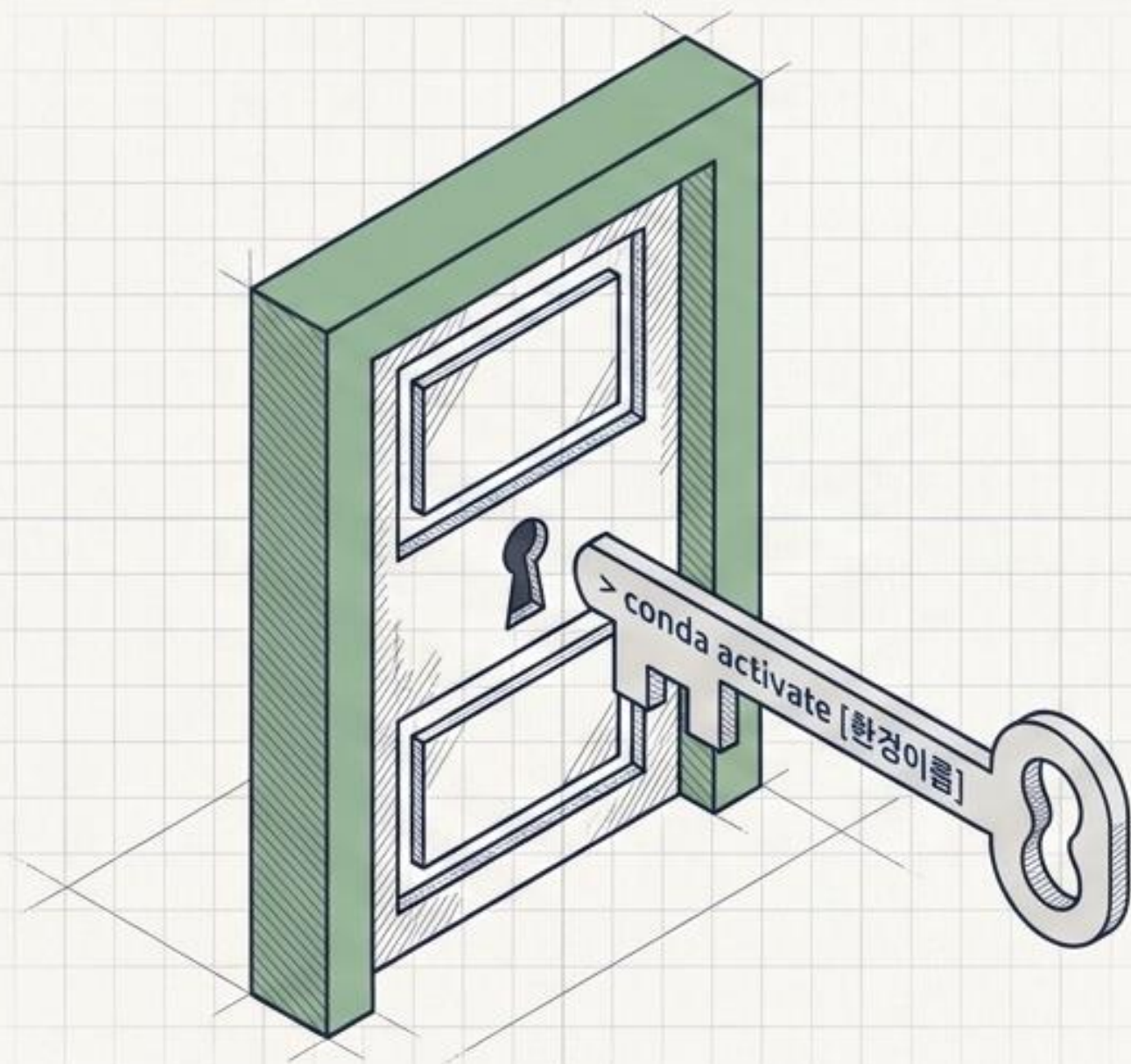
손쉬운 리셋

실습하다가 라이브러리가 꼬이면?
맥북 전체를 고칠 필요 없이 이 '방'만 지우고 10초 만에 다시 만들면 됩니다.



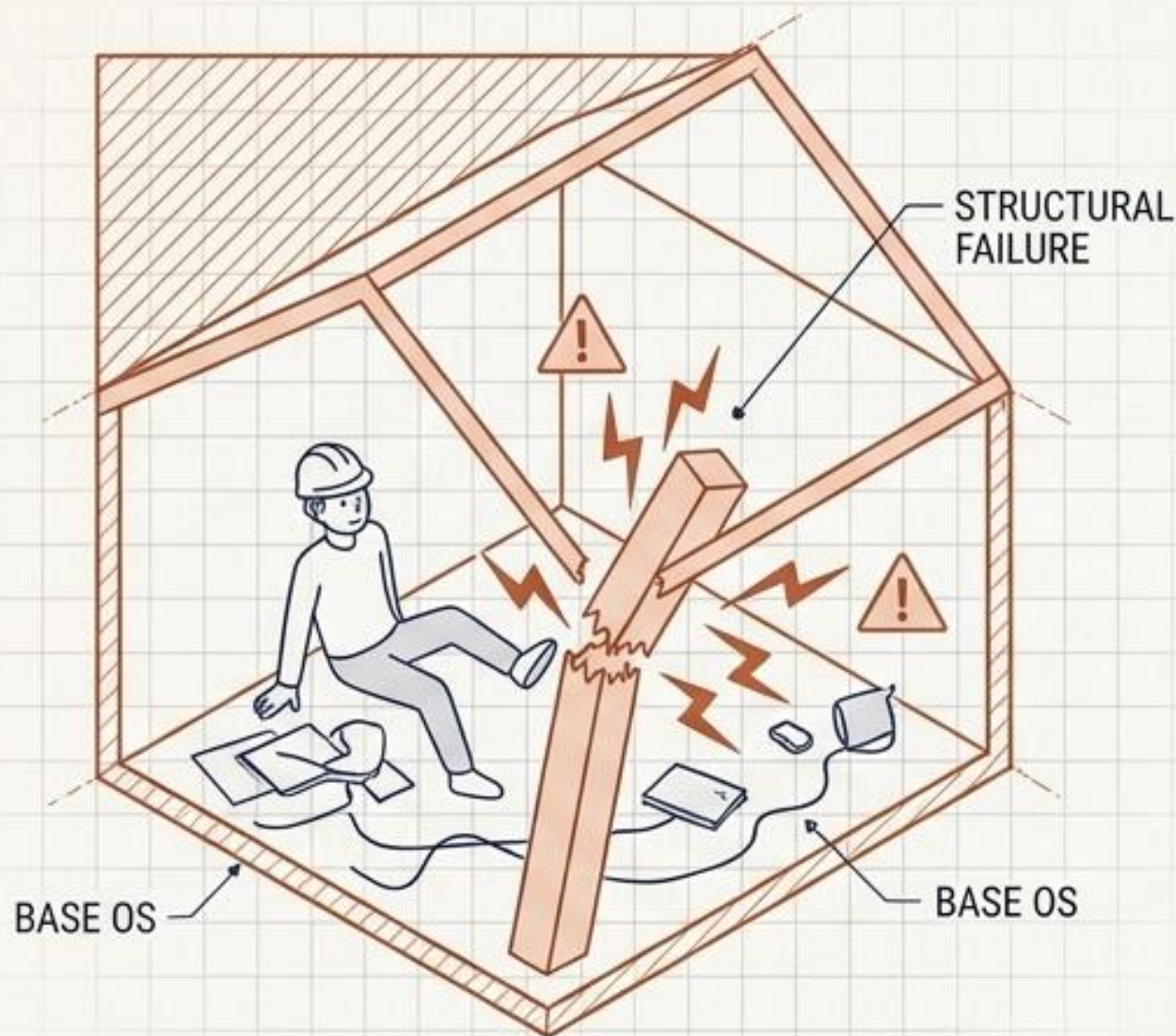
방을 만들었다고 끝이 아닙니다. 문을 열어야 합니다.

미니콘다를 설치하고 방을 만들었다고 자동으로 실습 준비가 끝나는 것이 아닙니다. 방안에 들어가지 않으면 파이썬이나 판다스 같은 도구를 꺼내 쓸 수 없습니다. `conda activate`는 굳게 닫힌 ‘수업 전용 방 방을 문을 열고 안으로 들어간다’는 명시적 선언입니다.



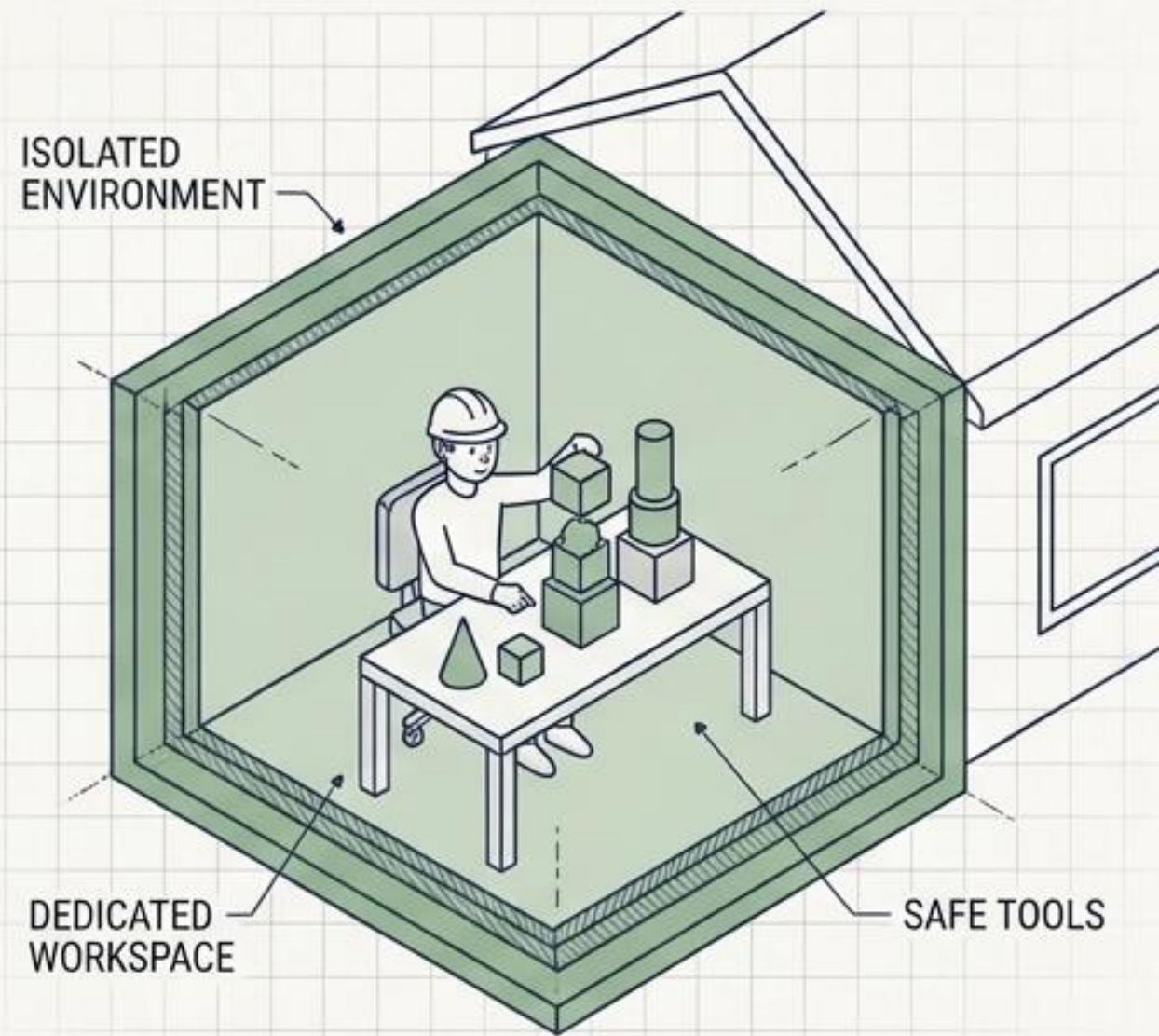
왜 반드시 터미널을 열고 환경 안으로 들어가야 할까?

[문을 열기 전 - 거실]



문을 열고 들어가지 않은 상태에서 코딩하면, 맥북 자체의 엉뚱한 기본 도구를 건드려 시스템 전체가 꼬일 위험이 있습니다.

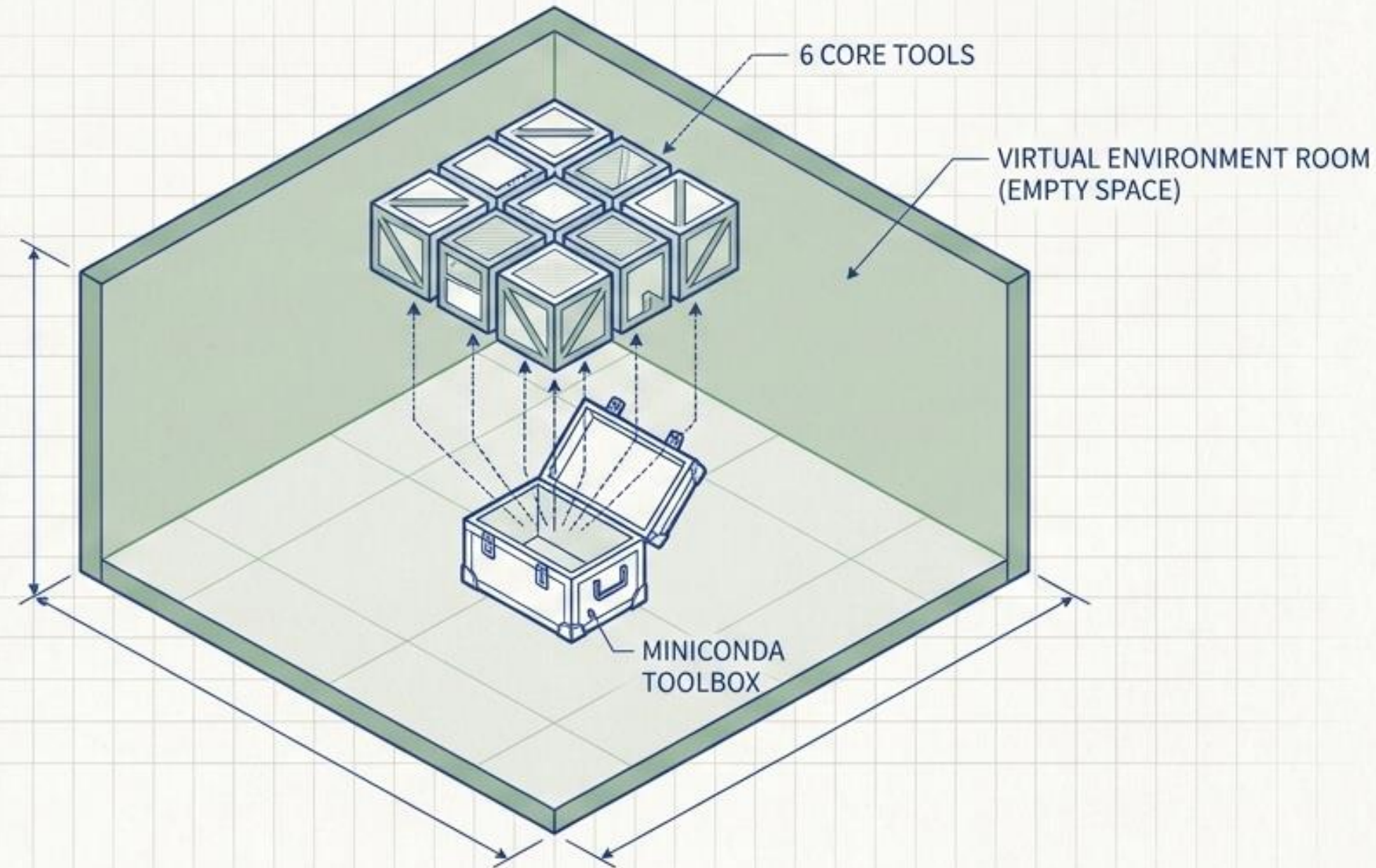
[문을 연 후 - 실습실]



내가 안전하게 격리해 둔 실습실 책상 앞에 앉아, 오직 그 방에 있는 도구만 사용하여 안전하게 작업합니다.

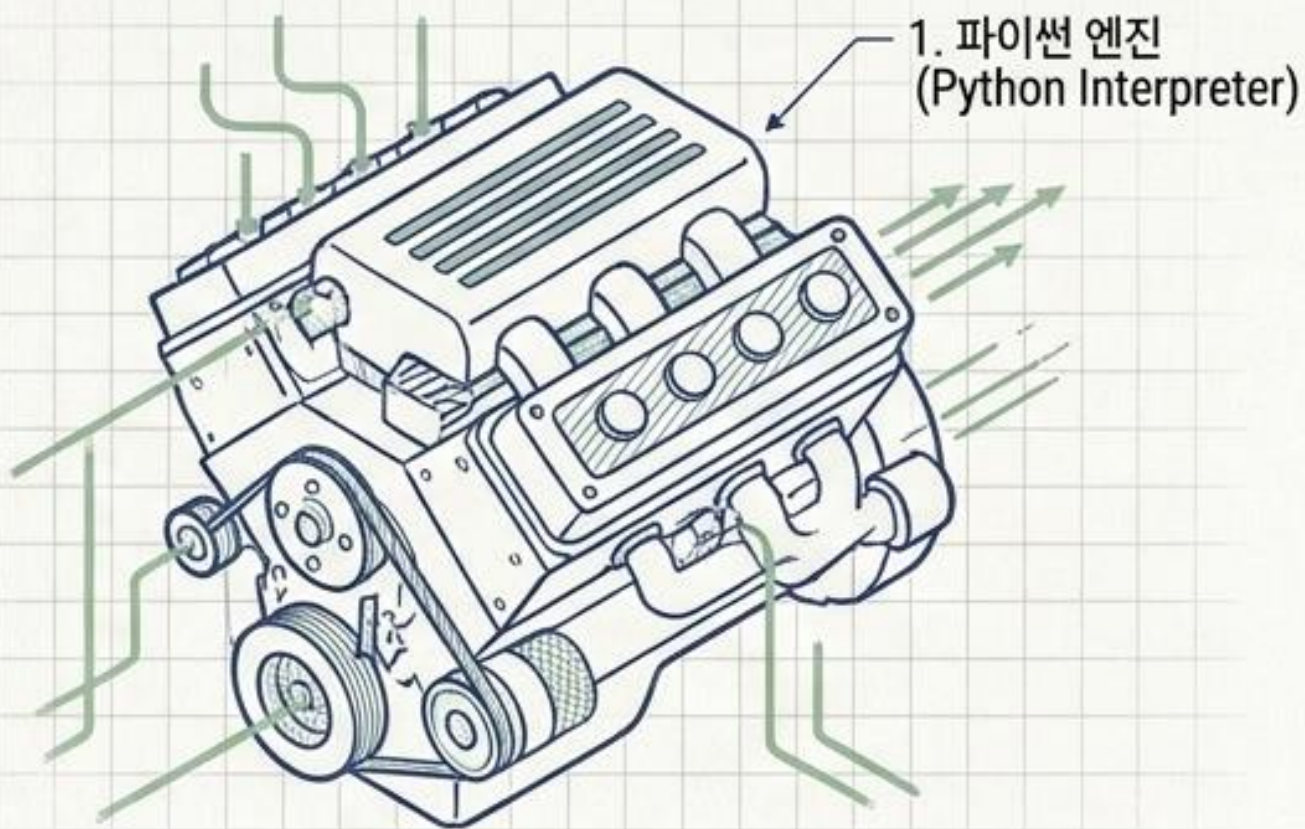
미니콘다의 기본 공구함 열기 (Unboxing)

미니콘다는 가장 가벼운 빈 방을 제공하지만, 방 안에서 스스로 작동하고 외부에서 다른 가구를 안전하게 들어오기 위한 ‘최소한의 6가지 핵심 공구’는 기본적으로 세팅해 둡니다.



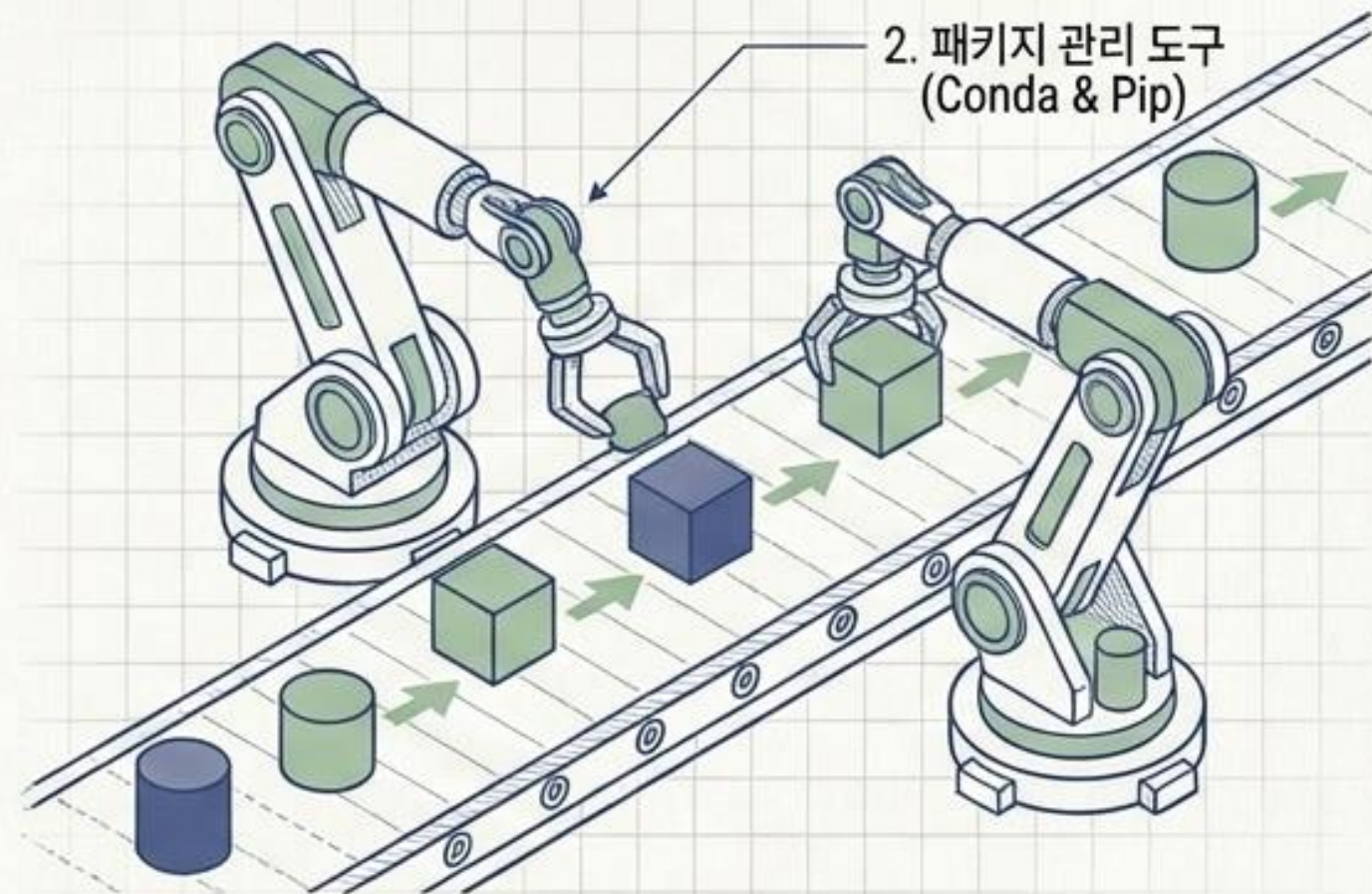
핵심 엔진과 패키지 관리자

1. 파이썬 엔진 (Python Interpreter)



python: 가상환경 전체를 돌리는 심장.
환경 생성 시 선택한 기본 파이썬 버전이 설치됩니다.

2. 패키지 관리 도구 (Conda & Pip)



conda: 가상환경을 관리하고 패키지를 설치하는
총괄 메인 관리자입니다.

pip, setuptools, wheel: 콘다 외에 파이썬 기본 저장소(PyPI)에서
라이브러리를 직접 가져와 조립하는 필수 도구들입니다.

외부 연결 보안망과 분석 엔진

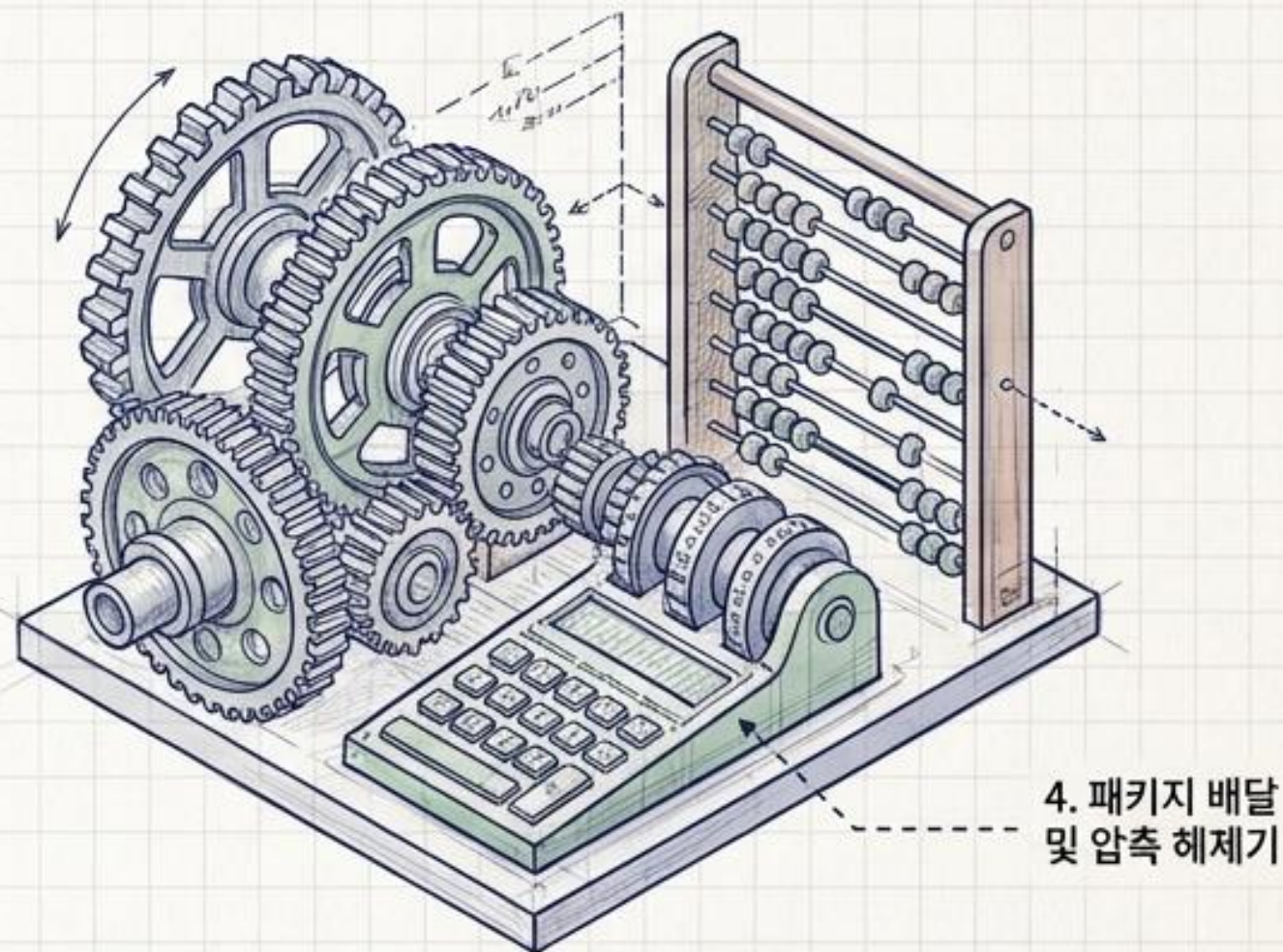
3. 인터넷 연결 및 보안 필터 (Security & Network)



3. 인터넷 연결 및 보안 필터

openssl, ca-certificates, certifi, cryptography: 외부 패키지를 HTTPS로 안전하게 다운받고 신뢰성을 검증하는 보안 라이브러리입니다.

4. 패키지 배달 및 압축 해제기 (Conda Internal Engines)



4. 패키지 배달 및 압축 해제기

conda-libmamba-solver: 패키지 간의 복잡한 버전을 빠르게 분석하고 충돌 여부를 계산하는 최신 고속 분석 엔진입니다.

conda-package-handling, streaming: 다운로드한 무거운 압축 파일(.conda)을 방 안에 안전하게 푸는 도구입니다.

시스템 소통 및 기초 연동 도구

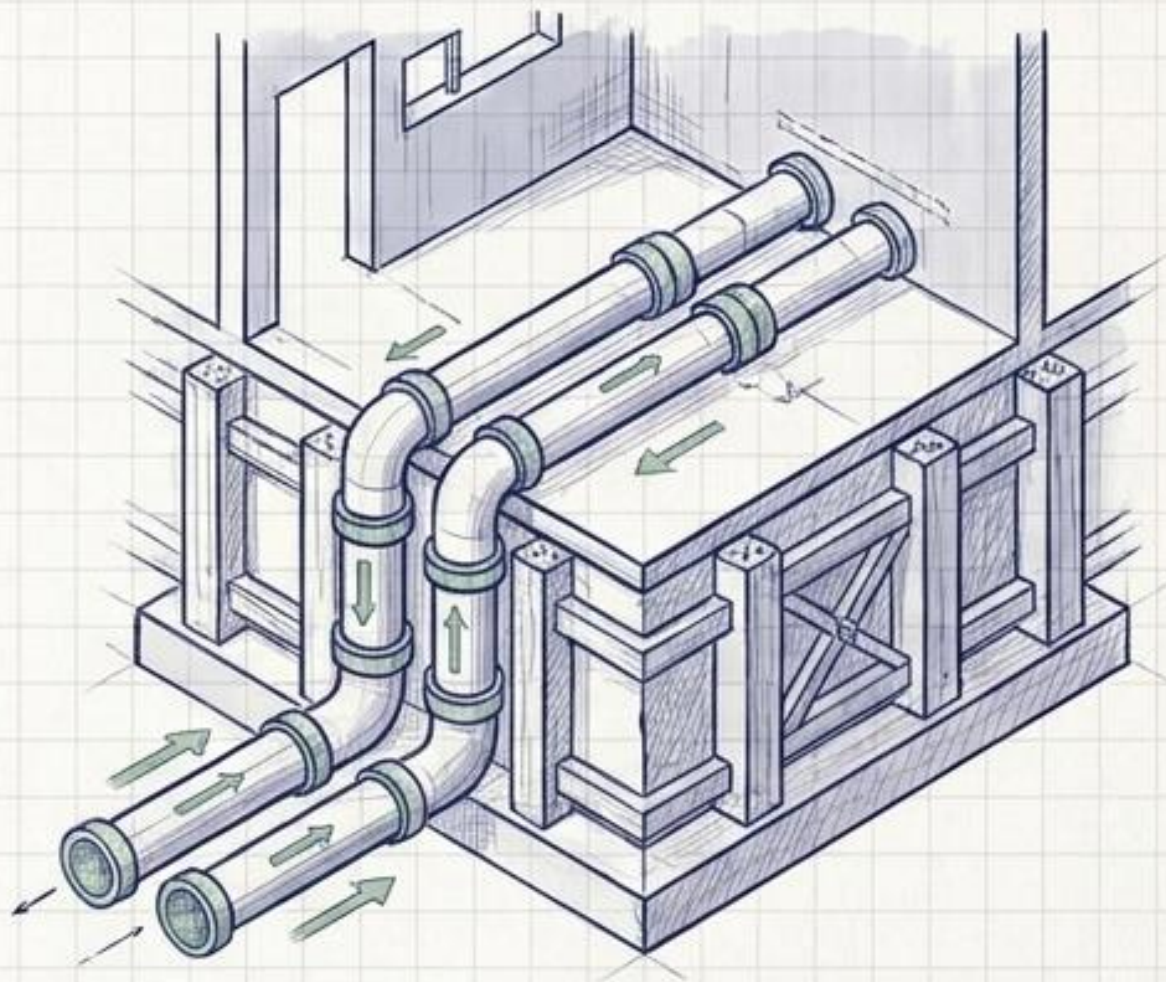
5. 터미널 및 콘솔 지원 도구 (CLI Tools)



5. 터미널 및 콘솔
지원 도구

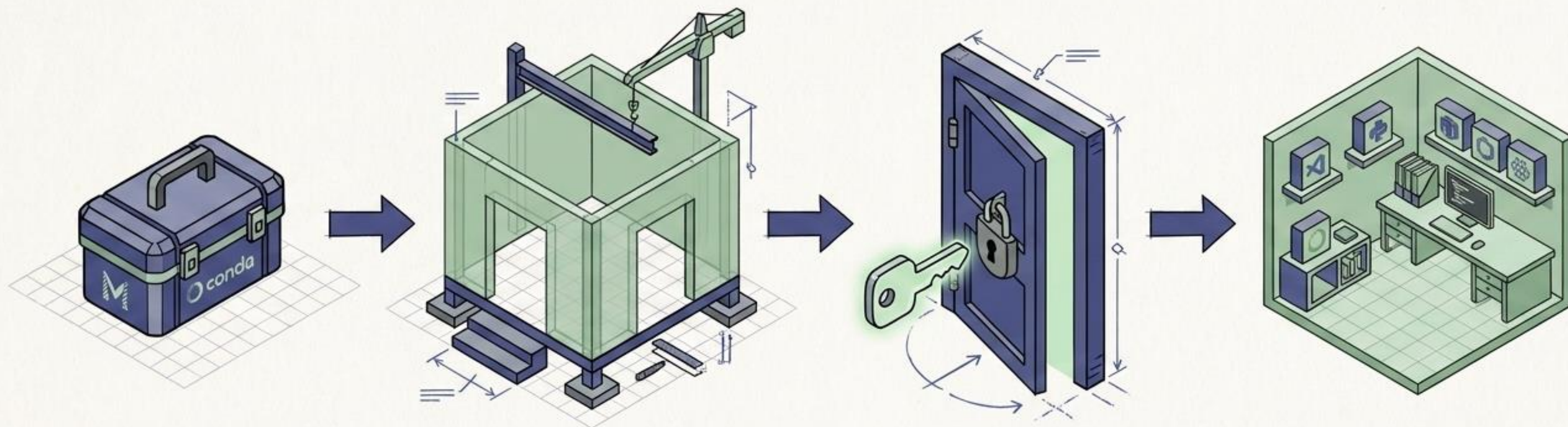
colorama, click: 흑백 터미널 화면에 색상을 입히고 가독성을 높여주는 기초 인터페이스 라이브러리입니다.

6. OS 및 기초 시스템 연동 라이브러리 (System Libraries)



sqlite, bzip2, zlib: 가상환경이 맥북(OS) 환경과 무리 없이 통신하고, 기초적인 데이터를 압축/저장하기 위한 필수 시스템 모듈입니다.

완벽한 워크플로우 멘탈 모델



도구의 선택: 가볍고 통제
가능한 공구함 준비
(Miniconda 설치)

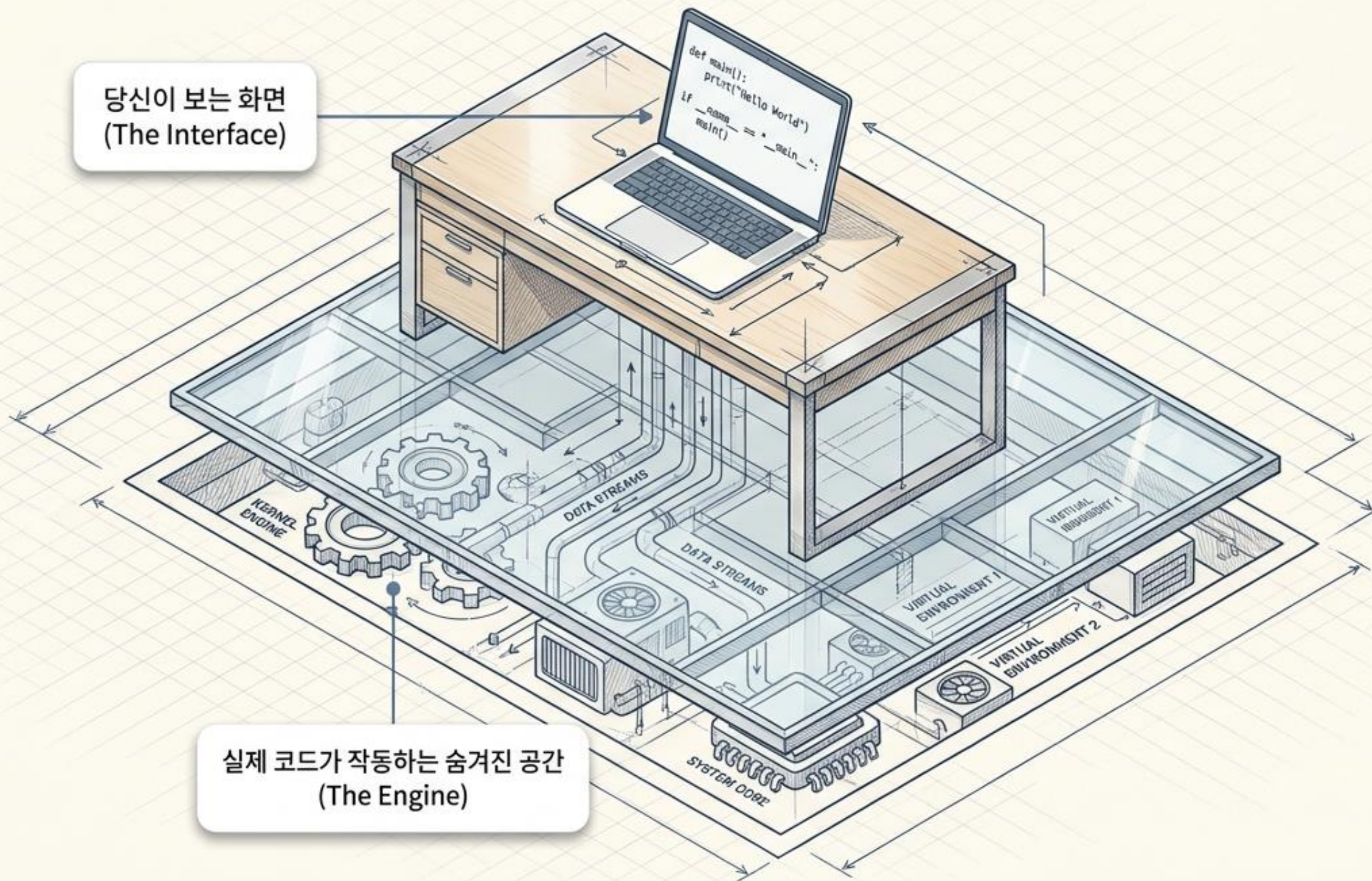
공간의 생성: 내 맥북을 보호할
완벽히 격리된 방 짓기
(conda create)

공간의 진입: 실습 전,
전용 방 문을 열고 입장하기
(conda activate)

안전한 개발:
6개의 핵심 공구와 함께
충돌 없는 코딩 시작

이제 여러분의 맥북을 완벽하게 보호하면서 코딩할 준비가 모두 끝났습니다.
터미널을 열고, 여러분만의 방으로 들어가세요.

보이지 않는 엔진: VS Code, 커널, 가상환경의 작동 원리



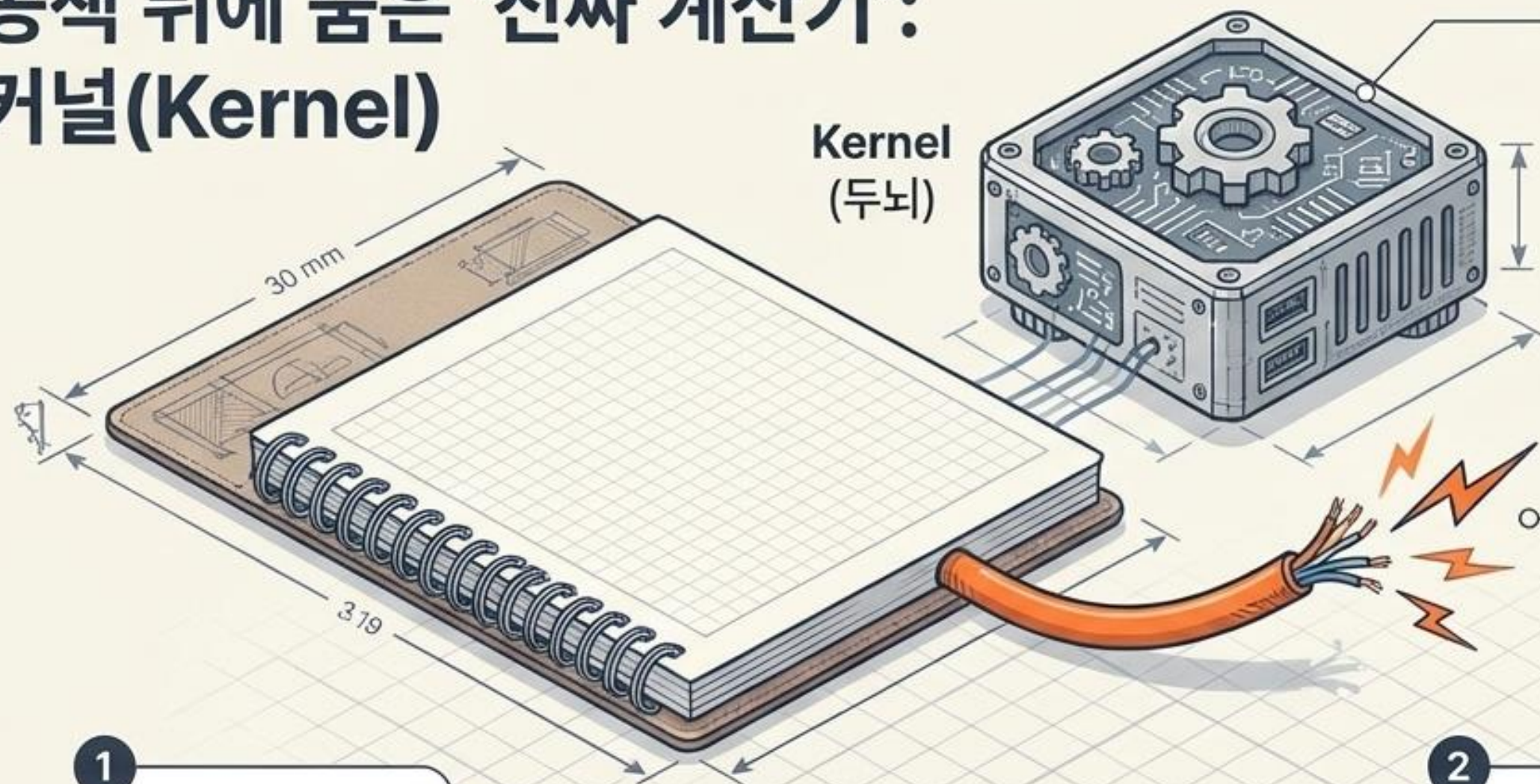
파일(.ipynb)을 여는 것은 그저 '예쁜 빈 공책'을 펼치는 행위입니다

VS Code의 한계:
작업 폴더를 지정하고
노트북 파일을
더블클릭하는 단계는
필기할 준비를
마친 것일 뿐입니다.

스스로 코드를 계산하거나
실행할 능력이 전혀 없는 상태
(필기와 낙서만 가능)



공책 뒤에 숨은 '진짜 계산기': 커널(Kernel)



커널의 역할:
공책 뒤에서 실제로 적힌 코드를 한 줄씩 읽고 실행하여 연산 결과를 다시 공책에 보여주는 실행 주체입니다.

⚠ 이 선이 연결되지 않으면 코드는 절대 실행되지 않습니다.

1 [노트북 (공책)]

연결선을 통해 텍스트 전달

2 [커널: 코드 한 줄씩 읽기 및 계산]

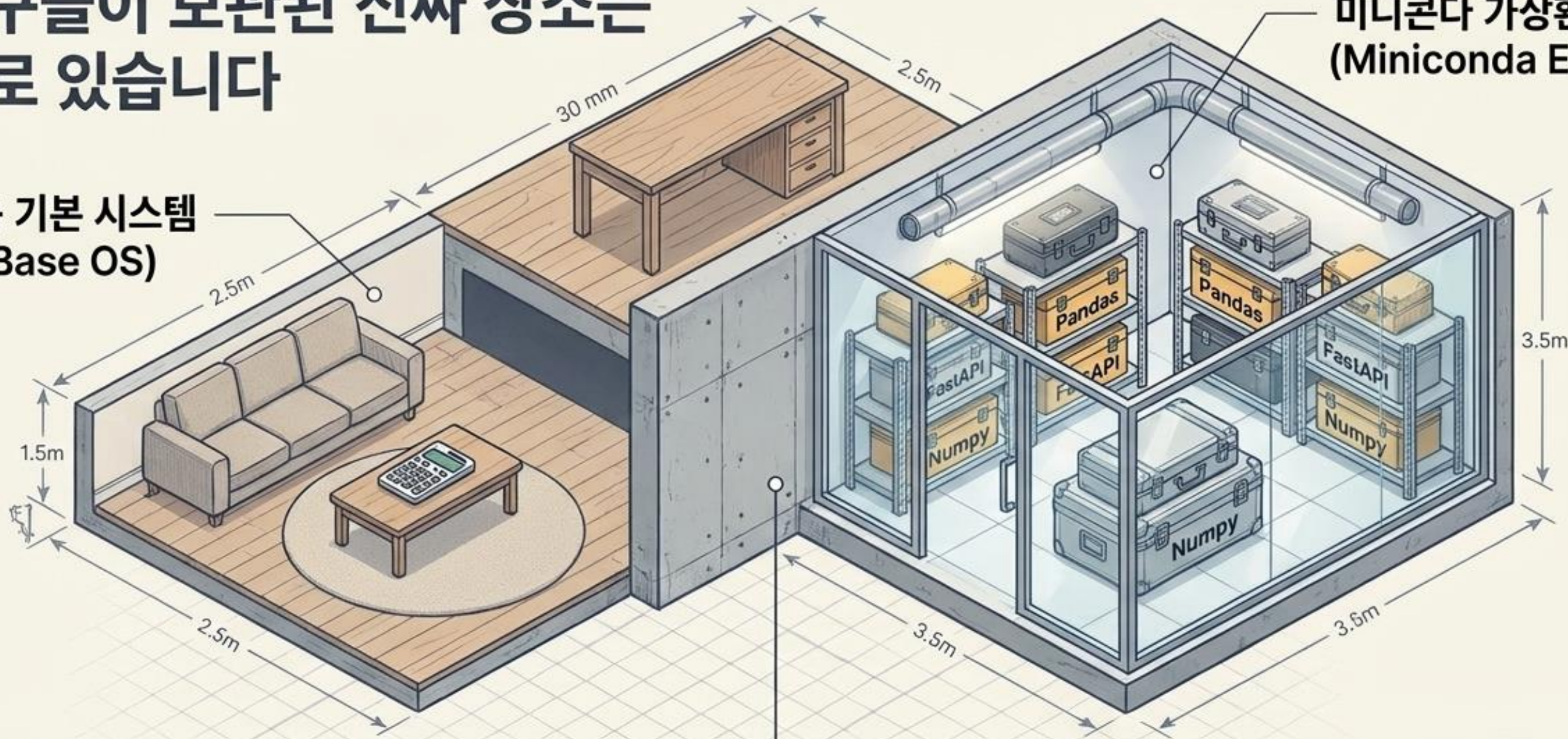
3 [노트북에 결과 출력]

연결선을 통해 결과 반환

도구들이 보관된 진짜 장소는 따로 있습니다

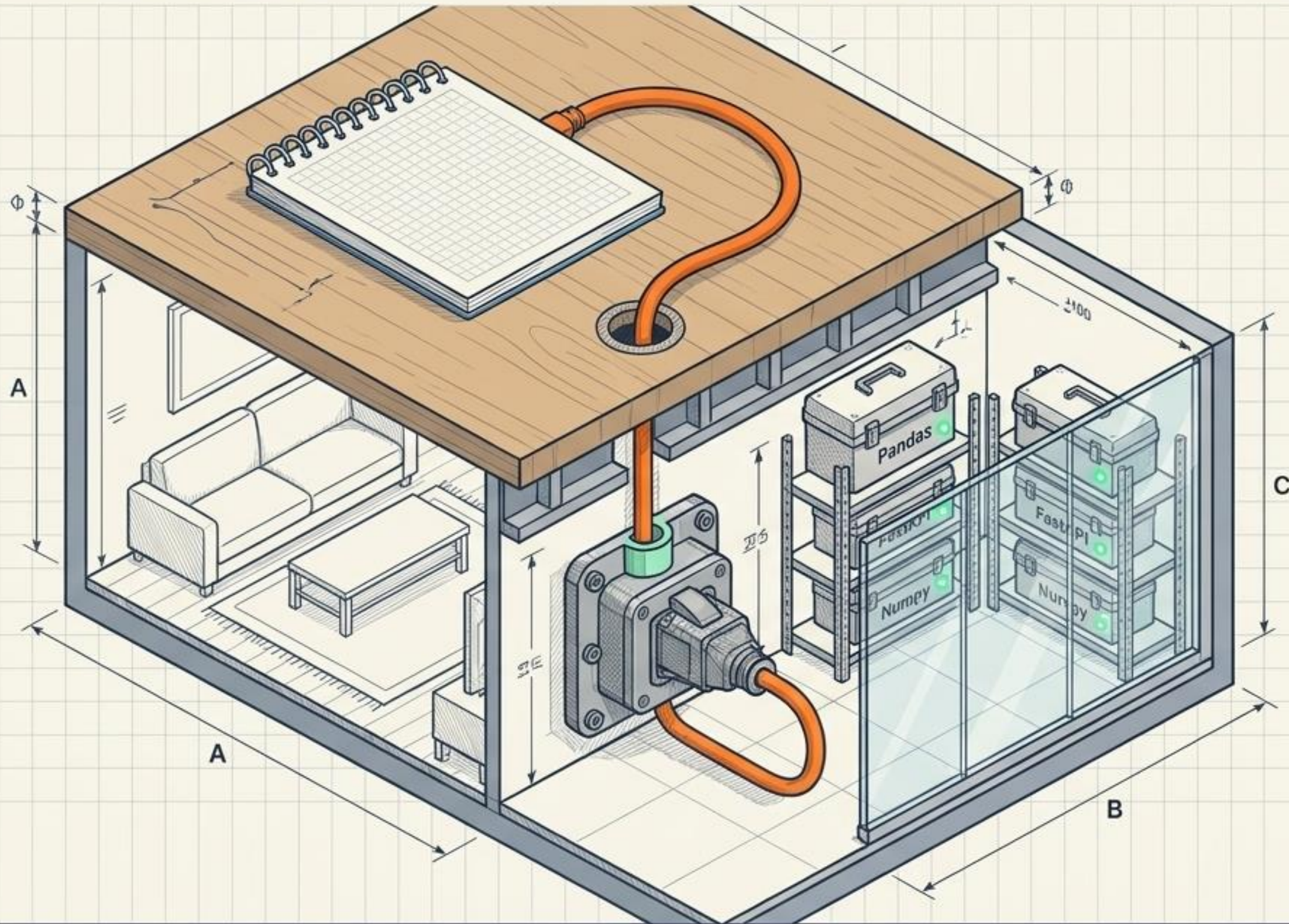
맥북 기본 시스템
(Base OS)

미니콘다 가상환경
(Miniconda Env)



가상환경의 개념: 우리가 코딩에 필요한 특수 도구(라이브러리)들을 안전하게 격리해서 모아둔 독립된 방입니다.

"커널을 설정한다" = 연결선을 올바른 방에 꽂는 작업



정확한 연결:
공책의 연결선(커널)을
거실(맥북 기본 시스템)이
아닌, 우리가 준비한
수업 전용 실습실 안의
파이썬 계산기에 똑바로
꽂아 연결하는 행동을
의미합니다.

"Pandas 모듈을 찾을 수 없습니다" 에러가 발생하는 진짜 이유

맥북 기본 시스템
(Base OS)

미니콘다 가상환경
(Miniconda Env)



연결의 오류: 방 안에 아무리 좋은 라이브러리를 설치해 두었어도,
연결선이 거실에 잘못 꽂혀 있으면 공책은 도구를 찾을 수 없습니다.

내 코드가 실행될 장소 비교하기

	거실 (맥북 기본 시스템)	수업 전용 실습실 (미니콘다 가상환경)
비유	일상적인 생활 공간 	안전하게 격리된 특수 연구실 
설치된 도구	기본 내장 프로그램만 존재 	Pandas, FastAPI 등 수업용 특수 도구 완비 
위험도	함부로 건드리면 시스템 전체가 망가질 위험 	언제든 지우고 다시 만들 수 있는 안전지대
판정	✗ 여기에 연결선을 꽂지 마세요	✓ 수업 중 반드시 이곳에 연결하세요 

완벽한 회로도: 코드가 안 돌아갈 때 확인해야 할 단 한 가지

학생들을 위한 황금 법칙 (The Golden Rule)

에러가 나거나 코드가 실행되지 않아 해맬 때는,
코드를 고치기 전에 이 공책의 연결선이 '미니콘다
방'에 제대로 꽂혀 있는지 가장 먼저 확인하세요.

1. VS Code 우측 상단 커널 상태 확인
2. '기본 시스템'이 아닌 'Miniconda'가
선택되었는지 체크

